

Cloud Cost Janitor

An AI agent that cleans up cloud waste, and knows when to stop. Agents That Act: TrueFoundry x Polaris Hackathon. Code: github.com/harshathghara/TrueFoundry_Hackathon

1. The problem

Cloud accounts accumulate idle spend: unattached volumes, stopped instances still paying for their disks, idle load balancers, unused Elastic IPs and stale snapshots. Finding, pricing and safely deleting them is tedious toil worth delegating, as long as nothing is deleted without a human's sign-off.

2. What the agent reaches

A real AWS account, through our own MCP server (*aws-janitor*, built on boto3): EC2 volumes, instances, snapshots, Elastic IPs, load balancers and, read-only, Route 53. AWS credentials live only in the MCP server; the model and the sandbox never see them.

3. Where it stops

Discovery, pricing and blast-radius checks run unattended. All five destructive tools pause in TrueForge until a human approves or denies them, after seeing the cost and blast radius. Behind that gate, the tools refuse anything not tagged *janitor-demo=true*, always refuse *env=prod* and dry-run EC2 deletes; the agent snapshots volumes before deleting them; and IAM allows stopping or deleting only demo-tagged resources.

4. Architecture

```
React dashboard -> FastAPI bridge (trueforge-sdk, SSE) -> TrueForge
TrueForge -> TrueFoundry AI Gateway (model)
              -> Daytona sandbox (runs the agent's Python)
              -> aws-janitor MCP server -> AWS
```

5. How TrueForge was used

TrueForge runs the agent loop; its HTTP API configures the agent, the MCP connector, the TrueFoundry AI Gateway model and the Daytona sandbox. The agent writes and runs Python in the sandbox to build a ranked teardown plan. Approval gates pause the turn, and our dashboard resumes it through the SDK. Every run is traced in TrueForge Sessions.

6. Real versus mocked

Real: seeded AWS resources, every tool call, sandbox runs, model calls and approved deletions (demo: \$24.17/month saved, \$3.60 declined, \$8.00 skipped as unsafe). **Mocked:** unit tests use moto; prices come from a static us-east-1 table; idleness is judged from resource state, not CloudWatch history.

7. Known limits

One region and one account per run; static prices; an in-memory audit log; an unauthenticated MCP server bound to localhost; deletions limited to demo-tagged resources by design; and the smaller model occasionally attempts a redundant delete, which is reported as failed rather than hidden.